

Neural Network Optimization Techniques

Purpose

The purpose of this project is to understand and build a multi-layer neural network and train it to classify hand-written digits into 10 classes (digits 0-9). You will also be implementing different optimization techniques like dropout, momentum, and learning_rate scheduling and the use of mini-batch gradient descent.

Objectives

Learners will be able to

- Learn the fundamental concept of gradient descent and explore variations like stochastic gradient descent and minibatch gradient descent.
- Gain insights into regularization techniques, including L2 regularization (weight decay) and dropout, to prevent overfitting.

Technology Requirements

- GPU environment (optional)
- Jupyter Notebook
- Python3 (Python 3.8 and above)
- Numpy
- Matplotlib

Directions

Accessing ZyLabs

You will complete and submit your work through zyBooks's zyLabs. Follow the directions to correctly access the provided workspace:

1. Go to the Canvas project, "**Submission: Neural Network Optimization Techniques Project**"

2. Click the “**Load Submission...in new window**” button.
3. Once in Zylabs, click the **green button** in the Jupyter Notebook to get started.
4. Review the directions and resources provided in the description.
5. When ready, review the provided code and develop your work where instructed.

Project Directions

In this project, we will be using the MNIST dataset that contains grayscale samples of handwritten digits of size 28 x 28. It is split into a training set of 60,000 examples and a test set of 10,000 examples. Since we plan to use mini-batch gradient descent, we can work with a larger dataset. You will also see the improved speed of minibatch gradient descent compared to project 2, where we used batch gradient descent.

Note: You can reuse some of the code you implemented for the following functions from project 2.

You need to work on the following tasks:

1. **ReLU (From project 2):** It is a piecewise linear function defined as

$$\text{ReLU}(Z) = \max(0, Z)$$

Hint: Use [numpy.maximum](#)

2. **ReLU - Gradient (From project 2):** The gradient of $\text{ReLU}(Z)$ is 1 if $Z > 0$ else it is 0.
3. **Linear activation and its derivative (from project 2):** There is no activation involved here. It is an identity function.

$$\text{Linear}(Z) = Z$$

4. **Softmax Activation and Cross-entropy Loss Function (from project 2):** Define a function to compute the softmax activation of the inputs Z and estimate the cross-entropy loss
5. **Derivative of the softmax_cross_entropy_loss.(from project 2):** Define a function that computes the derivative of the softmax activation and cross-entropy loss
6. **Dropout forward:** The dropout layer is introduced to improve regularization by reducing overfitting. The layer will zero out some of the activations in the input based on the 'drop_prob' value. Dropout is only applied in 'train' mode and not in 'test' mode. In the 'test' mode the output activations are the same as input activations. You need to implement the inverted dropout method we discussed in the lecture and define 'prob_keep' as the percentage of activations remaining after dropout, if drop_out = 0.3, then prob_keep = 0.7, i.e., 70% of the activations are retained after dropout.

7. **Dropout Backward:** In the backward pass, you need to estimate the derivative w.r.t the dropout layer. You will need the 'drop_prob', 'mask', and 'mode' which are obtained from the cache saved during the forward pass.
8. **BatchNorm forward:** Batchnorm scales the input activations in a minibatch to have a specific mean and variance allowing the training to use larger learning rates to improve training speeds and provide more stability to the training. During training, the input minibatch is first normalized by making it zero mean and scaled to unit variance, i.e., $(0, 1)$ normalized. The normalized data is then converted to have a mean (β) and variance (γ) , i.e., $(\beta, \gamma I)$ normalized. Here, β and γ are the parameters for the batchnorm layer which are updated during training using gradient descent. The original batchnorm paper was implemented by applying batchnorm before nonlinear activation. However, batchnorm is more effective when applied after activation. The batchnorm implementation is tricky, especially the backpropagation. You may need to use the following source for reference: [Batchnorm backpropagation Tutorial](#). Note: The tutorial represents data as a (m, n) matrix, whereas we represent data as a (n, m) matrix, where n is feature dimensions and m is number of samples. If you are unable to implement the batchnorm correctly, you can still get the network to work by setting the variable 'bnorm_list = [0, 0, ..., 0, 0]'. This is a list of binary variables indicating if batchnorm is used for a layer (0 means no batchnorm for the corresponding layer). This variable is used in the 'multi_layer_network(.)' function when initializing the network.
9. **Batchnorm Backward:** The forward propagation for batchnorm is relatively straightforward to implement. For the backward propagation to work, you will need to save a set of variables in the cache during the forward propagation. The variables in your cache are your choice. The test case only tests for the derivative.
10. **Parameter Initialization:** We will define the function to initialize the parameters of the multi-layer neural network. The network parameters will be stored as dictionary elements that can easily be passed as function parameters while calculating gradients during backpropagation. The parameters are initialized using Kaiming He Initialization (discussed in the lecture). For example, in a layer with weights of dimensions (n_{out}, n_{in}) , the parameters are initialized as:

$$\omega = np.random.randn(n_{out}, n_{in}) * (2. / np.sqrt(n_{in})) \text{ and } b = np.zeros((n_{out}, 1)).$$

The dimensions for the weight matrix for layer $(l+1)$ are given by (Number_of_neurons_in_layer $_{(l+1)}$ x Number_of_neurons_in_layer $_l$). The dimension of the bias for layer $(l+1)$ is (Number_of_neurons_in_layer $_{(l+1)}$ X 1)

11. **Adam(momentum) Parameters - Velocity and Gradient Squares Initializations:** We will optimize using Adam (momentum). This requires velocity parameters V and Gradient-Squares parameters G . Here is a quick recap of Adam's optimization,

$$V_{t+1} = \beta V_t + (1 - \beta) \nabla J(\theta_t)$$

$$G_{t+1} = \beta_2 G_t + (1 - \beta_2) \nabla J(\theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{G_{t+1} + \epsilon}} V_{t+1}, \theta \in \{W, b\}$$

Parameters V are the momentum velocity parameters and parameters G are the Gradient-Squares. $\nabla J(\theta)$ is the gradient term dW or db , and $\nabla J(\theta)^2$ is the element-wise square of the gradient. α is the step_size for gradient descent. It has been estimated by decaying the 'learning_rate' based on the 'decay_rate' and 'epoch' numbers. β, β_2 and ϵ are constants that we will set up later. Each parameter and b 's for all the layers will have their corresponding velocity (V) and Gradient-Squares (G) parameters.

12. **Forward Propagation through a single layer (from project 2):** If the vectorized input to any layer of a neural network is A_{prev} and the parameters of the layer are given by (W, b) the output of the layer (before the activation is):

$$Z = W.A_{prev} + b$$

13. **Forward Propagation through a layer (linear → activation → batchnorm → dropout):** The input to the layer propagates through the layer in the order linear → activation → batchnorm → dropout saving different cache along the way.
14. **Multi-Layers forward propagation:** Starting with the input 'A0' and the first layer of the network, we will propagate A0 through every layer using the output of the previous layer as input to the next layer. we will gather the caches from every layer in a list and use it later for backpropagation. We will use the 'layer_forward(.)' function to get the output and caches for a layer.
15. **Backward Propagation for the linear computation of a layer (from project 2)**
16. **Back Propagation through a layer (dropout → batchnorm → activation → linear):** We will define the backpropagation for a layer. We will use the backpropagation for the dropout, followed by backpropagation for batchnorm, backpropagation of activation, and backpropagation of a linear layer, in that order. This is the reverse order of the forward propagation.
17. **Multi-layer Back Propagation:** We have defined the required functions to handle backpropagation for a single layer. Now we will stack the layers together and perform back propagation on the entire network starting with the final layer. We will need the caches stored during forward propagation.
18. **Parameter Update Using Adam (momentum):** The parameter gradients (dW, db) calculated during backpropagation are used to update the values of the network parameters using Adam optimization which is the momentum technique we discussed in the lecture.

$$V_{t+1} = \beta V_t + (1 - \beta) \nabla J(\theta_t)$$

$$G_{t+1} = \beta_2 G_t + (1 - \beta_2) \nabla J(\theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{G_{t+1} + \epsilon}} V_{t+1}, \theta \in \{W, b\}$$

Parameters V are the momentum velocity parameters and parameters G are the Gradient-Squares. $\nabla J(\theta)$ is the gradient term dW or db , and $\nabla J(\theta_t)^2$ is the element-wise square of the gradient. α is the step_size for gradient descent. It has been estimated by decaying the learning_rate based on decay_rate and epoch number.

19. **Prediction:** This is the evaluation function that will predict the labels for a minibatch of input samples We will perform forward propagation through the entire network and determine the class predictions for the input data
20. **Training** We will now initialize a neural network with 3 hidden layers whose dimensions are 100, 100, and 64. Since the input samples are dimension 28×28 , the input layer will be dimension 784. The output dimension is 10 since we have a 10-category classification. We will train the model compute its accuracy on both training and test sets and plot the training cost (or loss) against the number of iterations.

Note: Most of the functions for the steps above are provided for you in your notebook to make it a little easier.

Submission Directions for Project Deliverables

Learners are expected to work on the project individually. Ideas and concepts may be discussed with peers or other sources can be referenced for assistance, but the submitted work must be entirely your own.

You must complete and submit your work through zyBooks's zyLabs to receive credit for the project:

1. To get started, use the provided Jupyter Notebook in your workspace.
2. All necessary datasets are already loaded into the workspace.
3. Execute your code by clicking the **"Run"** button in top menu bar.
4. When you are ready to submit your completed work, click on **"Submit for grading"** located on the bottom left from the notebook.
5. You will know you have completed the project when feedback appears below the notebook.
6. If needed: to resubmit the project in zyLabs
 - a. Edit your work in the provided workspace.
 - b. Run your code again.

- c. Click “**Submit for grading**” again at the bottom of the screen.

Your submission score will automatically be populated from zyBooks into your course grade. However, the course team will review submissions after the due date has passed to ensure grades are accurate.

Evaluation

This project is auto-graded. There are seventeen (17) test cases in total.

- Eleven (11) of the seventeen (17) are auto-graded.
- The remaining six (6) test cases will evaluate the functions from the “Multi-Category Neural Network Assignment” used in this assignment, with each worth 0 points.

Please review the notebook to see the points assigned for each test case. A percentage score will be passed to Canvas based on your score.